Arquitecturas y programación de procesadores gráficos

Nicolás Guil Mata

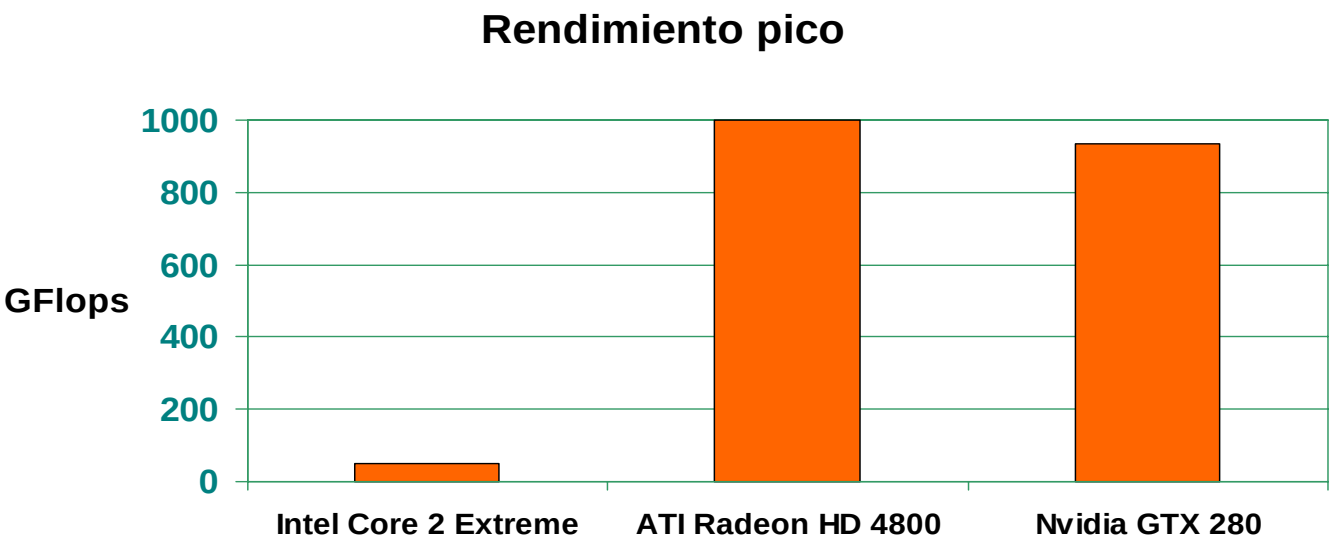
Dpto. de Arquitectura de Computadores

Universidad de Málaga

Indice

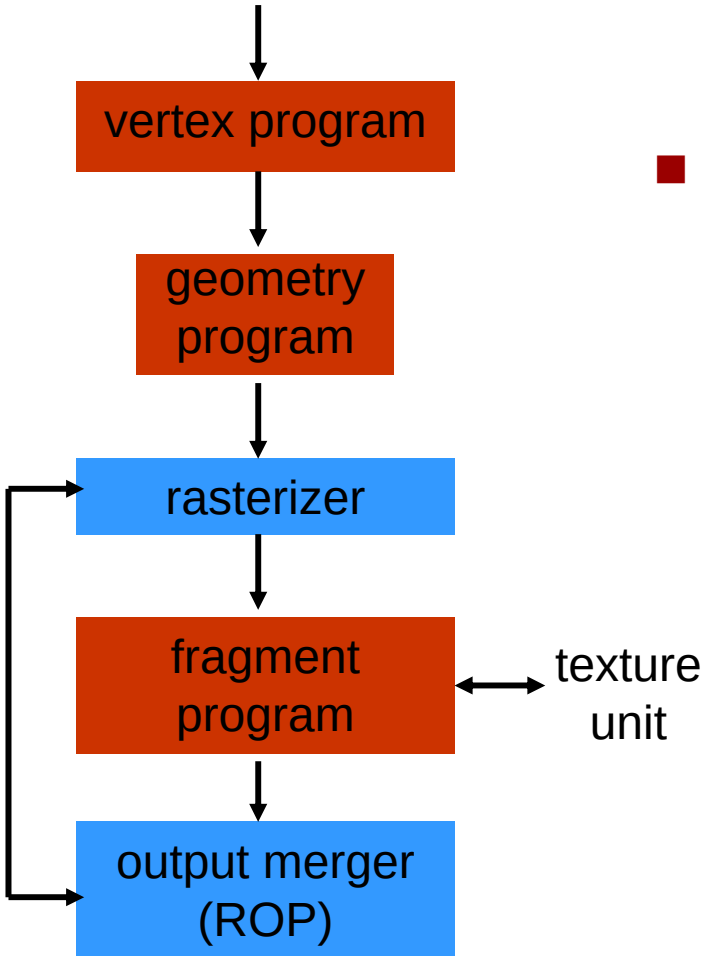
- Arquitectura de las GPUs
  - Arquitectura unificada
  - Nvidia: GT200
    - Diagrama de bloques
    - Memoria
    - ALU
    - Planificación de threads
- CUDA
  - Jerarquía de threads
  - Acceso a memoria global
  - Control de flujo
  - Herramientas para depurado y profiling
- Un ejemplo: multiplicación de matrices

Computación en GPUs



- Gran número de unidades funcionales
- Capaces de ejecutar miles de threads de forma concurrente
  - Ocultan latencias de acceso a memoria y riesgos
- Adecuado para kernels de computación (SPMD)
  - Acelerador gráfico y propósito general
- Jerarquía de memoria para soportar altos rendimientos

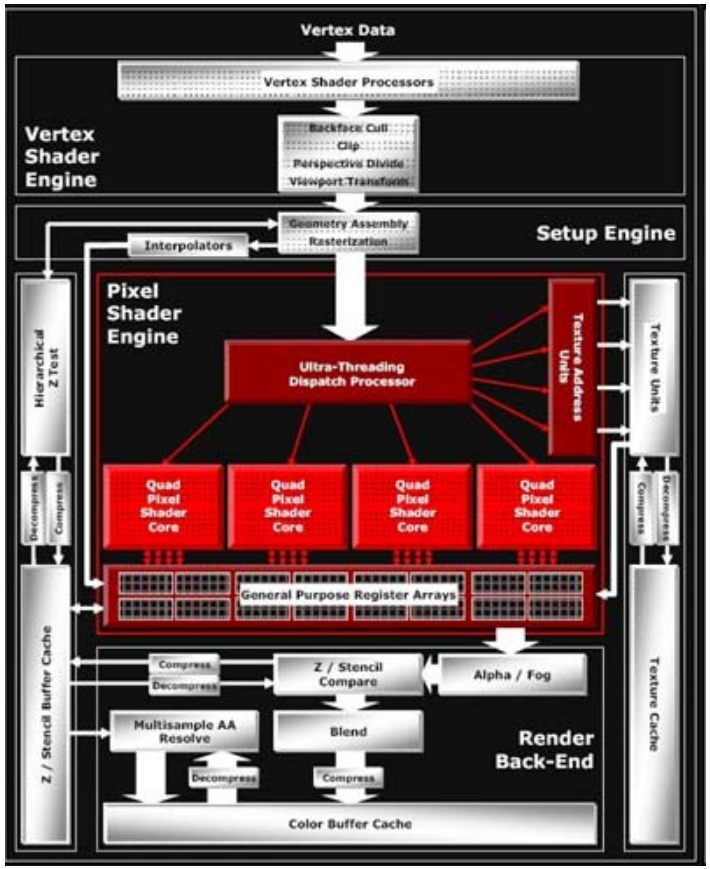
Programación de GPUs



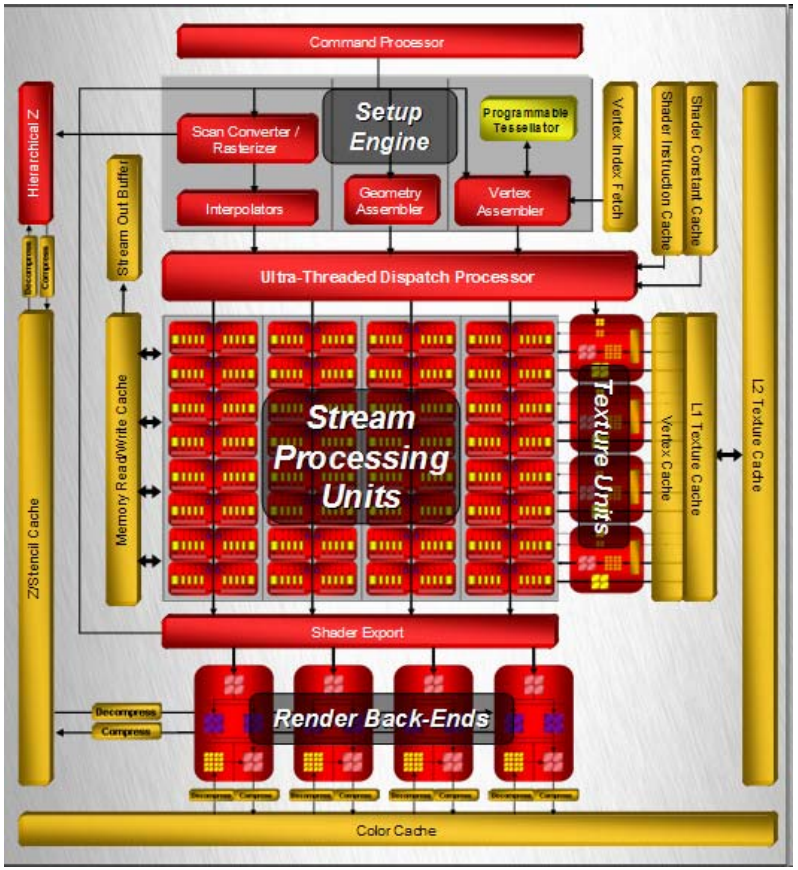
- Hardware de propósito específico para procesamiento gráfico
  - Rasterización, ROP
- Programación de la operaciones sobre vértices y píxeles
  - GLSL (3DLabs y OPenGL), Cg (Nvidia), HLSL (Microsoft)
  - CUDA (NVidia)
  - Sh (RapidMind), Brooks (Stanford)

Evolución de las GPUs

- Arquitectura unificada



ATI R520 (2005)

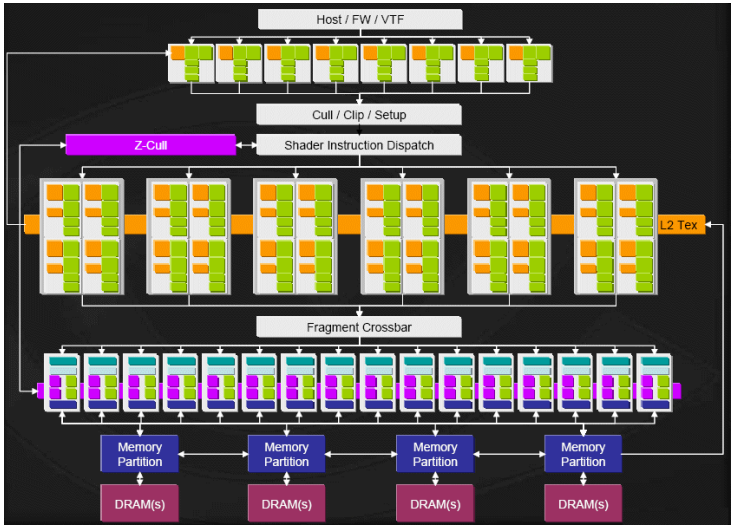


ATI R600 (2007)

Evolución de las GPUs

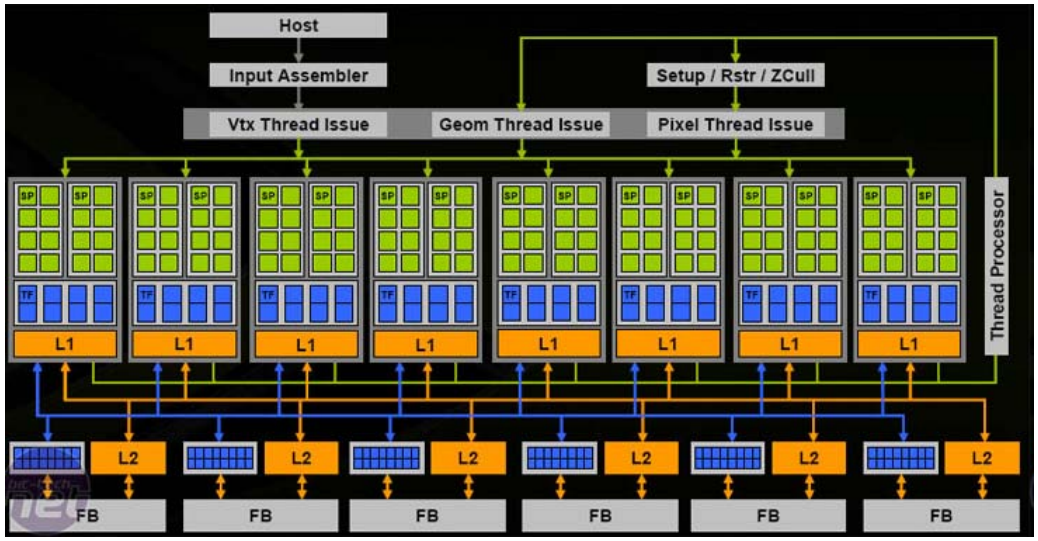
Nvidia G70

(2005)

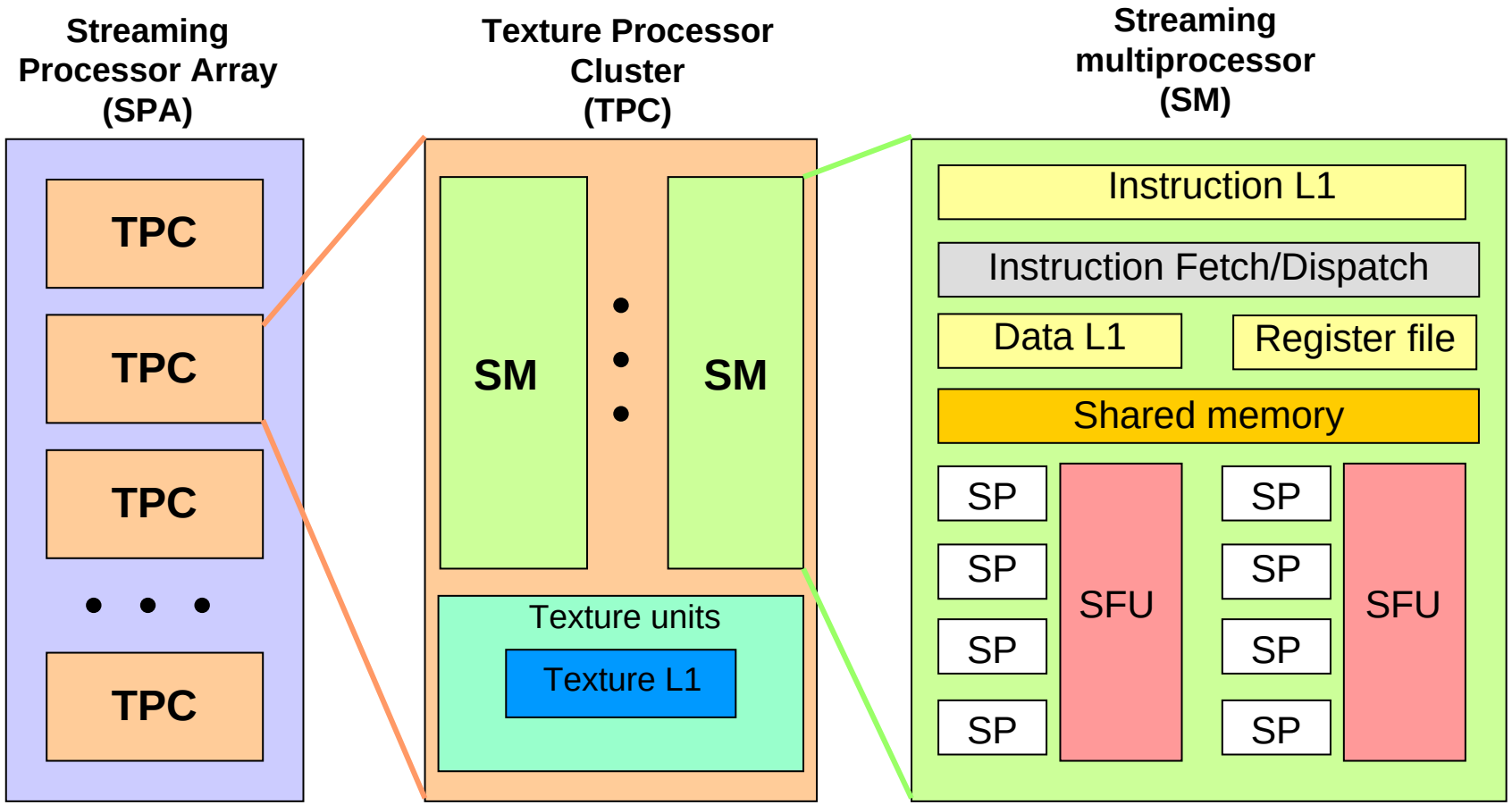


Nvidia G80

(2006)



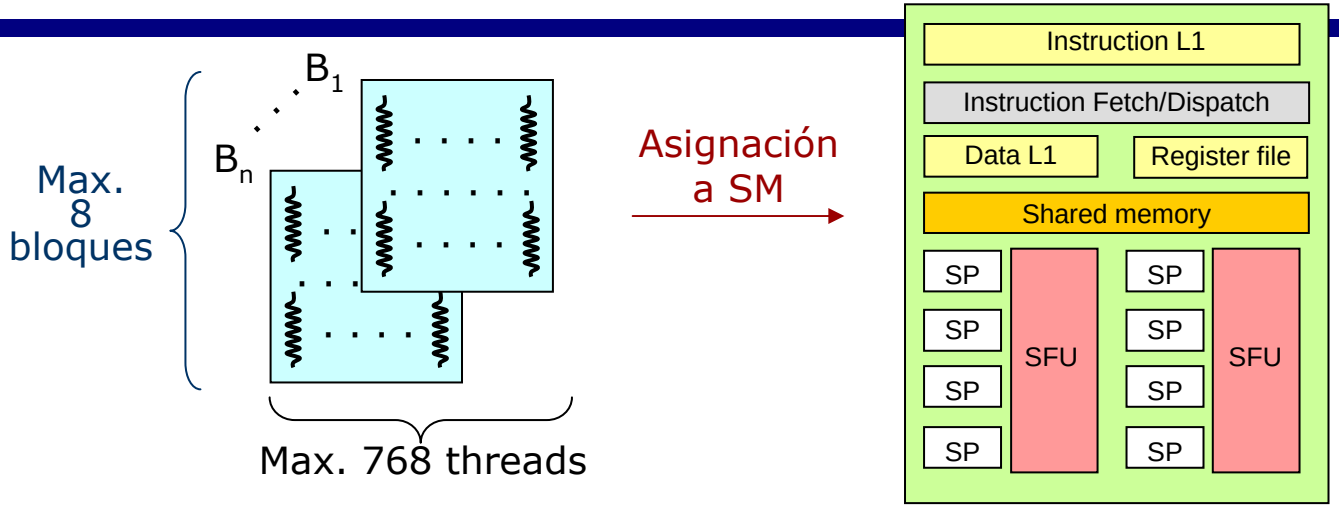
Arquitectura G80/G90/GT200



Arquitectura G80

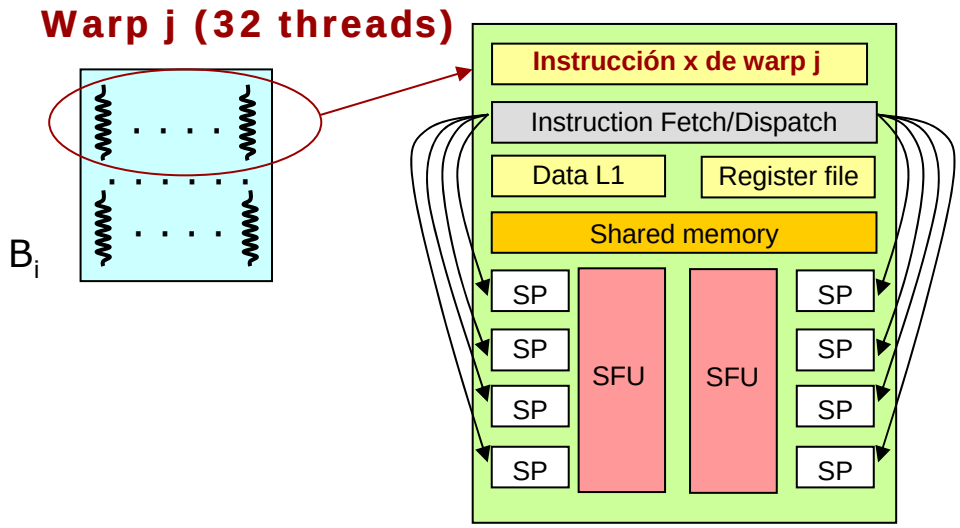
- Número variable de TPCs y SMs/TPC en las implementaciones de la arquitectura
- Memoria de textura de los TPCs para aplicaciones gráficas
- Planificación y ejecución de instrucciones en SM
  - Número máximo de threads asignados a un SM: 768
- Memoria cache de datos constantes
- 8192 registros (32 bits) partionados entre los threads asignados
- 16 KB de memoria local (más pequeña que el banco de registros)
  - Particionamiento explícito de la memoria local entre los bloques asignados a un SM
    - La memoria global es particionada entre los bloques asignados a cada SM y cada bloque sólo accede a su partición

Bloques de threads



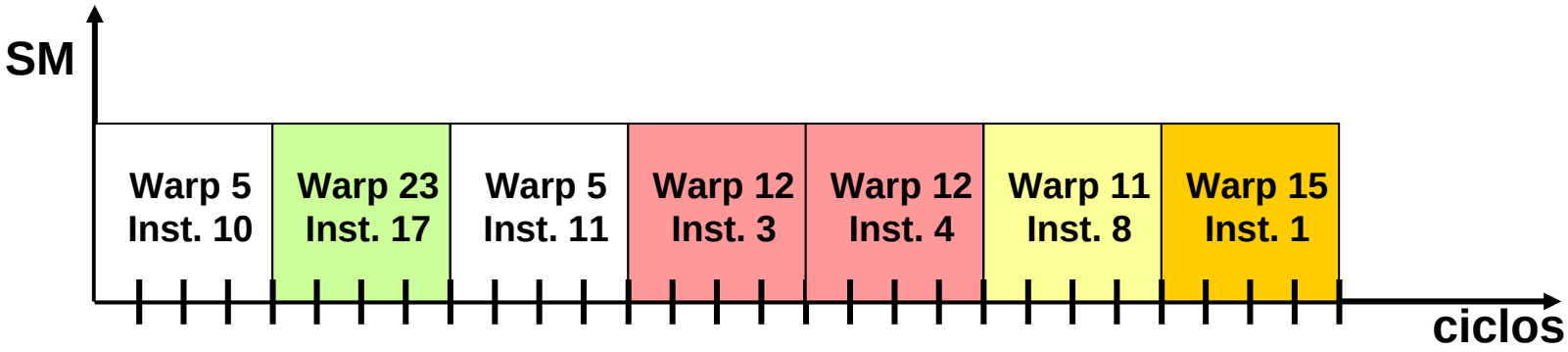
- A un SM se pueden asignar, como máximo, 768 threads.
- Los threads son asignados a los SMs en grupos llamados bloques (máximo 8 bloques por SM)
  - El bloque es la **unidad de asignación** de threads
    - Primitivas de sincronización (barrier) para todos los threads de un bloque
    - Todos los threads de un bloque pueden intercambiar información a través de la memoria compartida
  - Cada bloque puede contener 512 threads

Planificación de threads



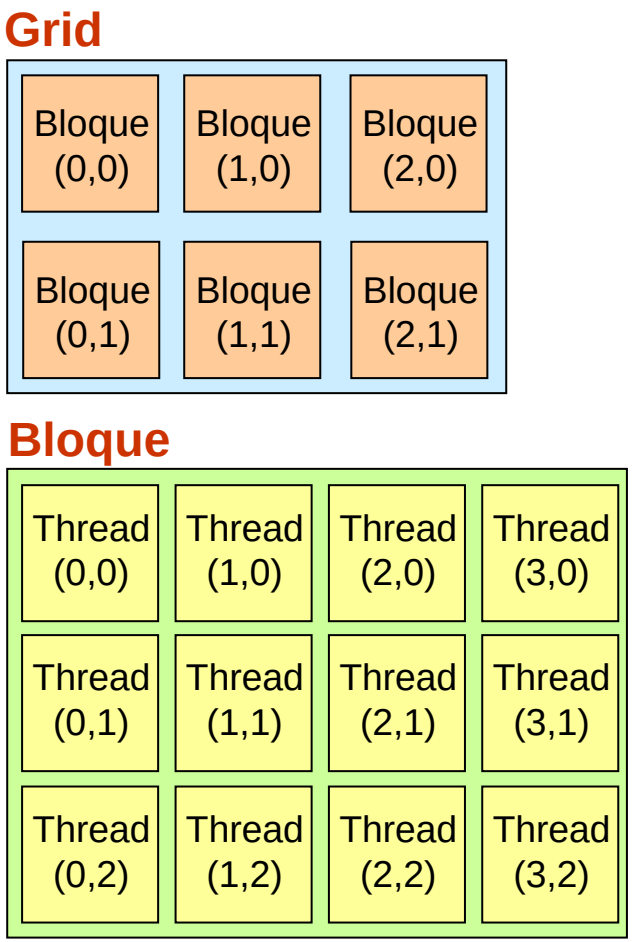
- SIMT (Single Instruction Multiple Thread): Cada instrucción del programa es ejecutada por 32 threads (warp) consecutivos del bloque
  - El warp es la **unidad de planificación**
  - Uso de round-robing/aging para seleccionar próximo warp a planificar de los disponibles en el pool de warps listos (operandos leidos)
- Ya que hay 8 SPs por SM, se tardan cuatro ciclos en ejecutar una instrucción de un warp.

Cambios de contexto



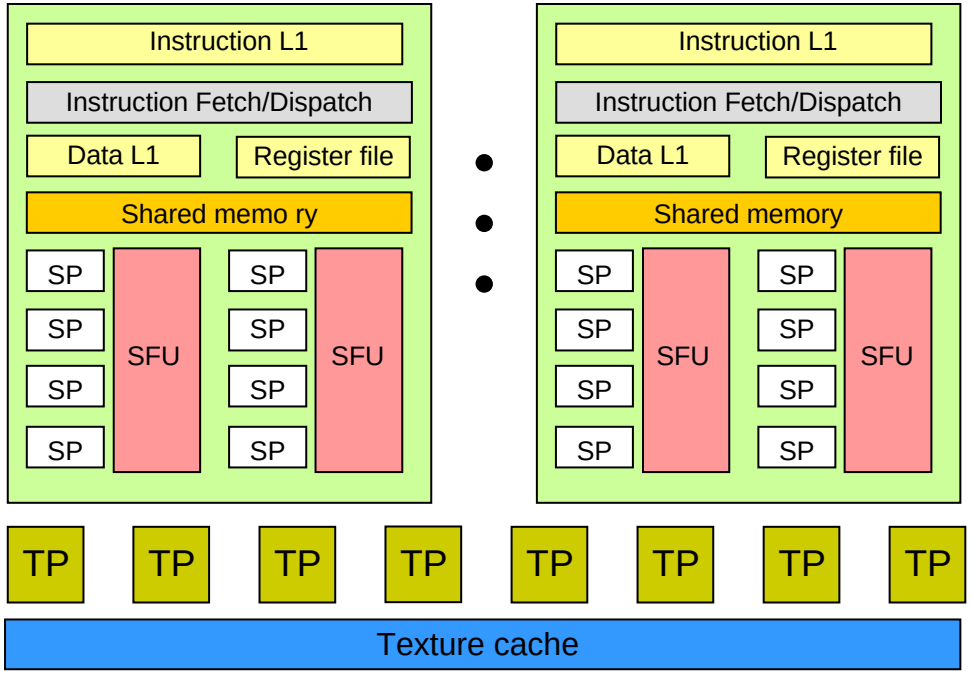
- Cambio de contexto sin coste en ciclos
  - probablemente ocultado por la latencia de ejecución de un warp
- La selección del próximo warp a ejecutar se basa en análisis de dependencias de instrucciones
  - Uso de la técnica del marcador (scoreboarding) para evitar riesgos

Jerarquía de thread



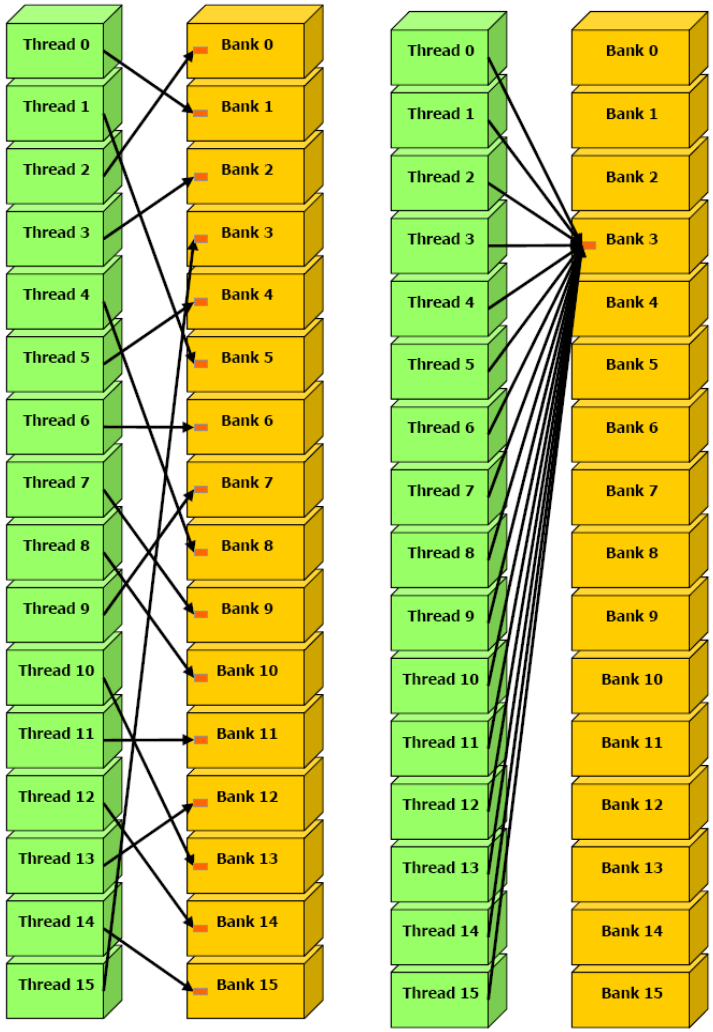
- Grid:
  - Cada kernel de ejecución del programa se mapea en un grid
    - Explicitar número de bloques en el grid
- Bloque:
  - Un bloque está compuesto de threads
    - Explicitar número de threads del bloque
  - Threads pueden cooperar compartiendo datos de la *shared memory*
  - Threads pueden sincronizarse (barrier)
- Warp:
  - Grupo de 32 threads consecutivos de un bloque
  - Tamaño de warp implícito

Memoria on chip



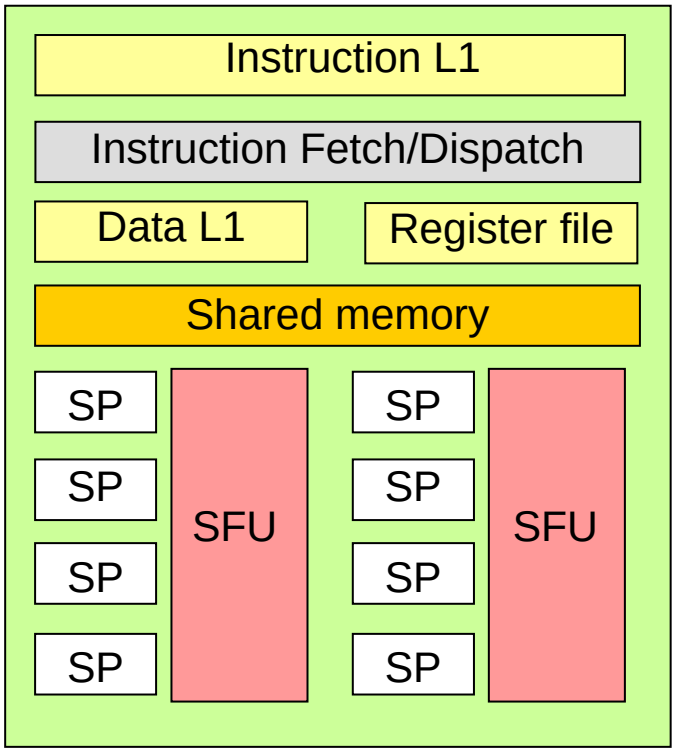
- Cada SM tiene
  - Fichero de registros particionado entre los threads asignados (R/W)
    - 8192 registros de 32 bits
    - 4 operandos por ciclo
  - Memoria compartida que permite intercambio de datos entre threads ejecutándose en el mismo SM
    - Manejo explícito (como cache de datos)
    - 16 Kbytes
    - No se garantiza compartición de datos entre threads de distintos bloques
  - Cache de datos constante (R)
    - Constantes leidas de la memoria constante
      - Primer acceso lento
    - Contenido de la cache es leido simultáneamente por los 8 SPs
  - Cache de texturas

Entrelazado de la SM



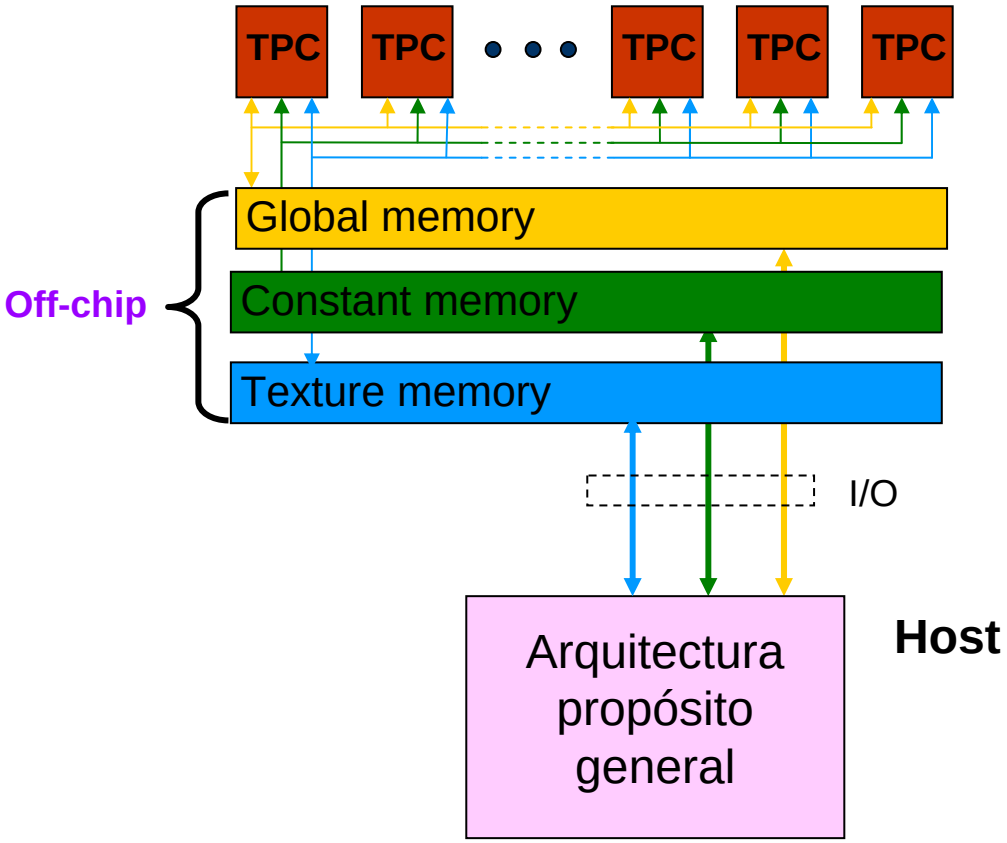
- 16 bancos de memoria
  - Entrelazado de orden inferior con palabra de 32 bits
- Acceso simultáneo de los threads de medio warp a memoria
  - Sin conflictos siempre que los threads accedan a módulos diferentes (cualquier permutación)
  - Si todos los threads acceden a la misma palabra de un banco específico

Unidades funcionales por SM



- 8 unidades funcionales segmentadas para operaciones de suma/multiplicación flotante (32 bits) y entera
- 2 super unidades funcionales segmentadas para operaciones flotantes complejas
  - Trigonométricas, inversa de raíz cuadrada, …
- Rendimiento pico en SM
  - G80
    - 18 operaciones flotantes por ciclo (8 mul/add flotante y 2 operaciones complejas)
    - En caso de 16 SM (1.35GHz)->388.8 GFlops
      - 388.8 = 16 x 18 x 1.35
  - GT200
    - 24 OPF por ciclo (1mul/add y 1 mul por SM)
    - 30 SM (1.35 Ghz) -> 972 GFlops

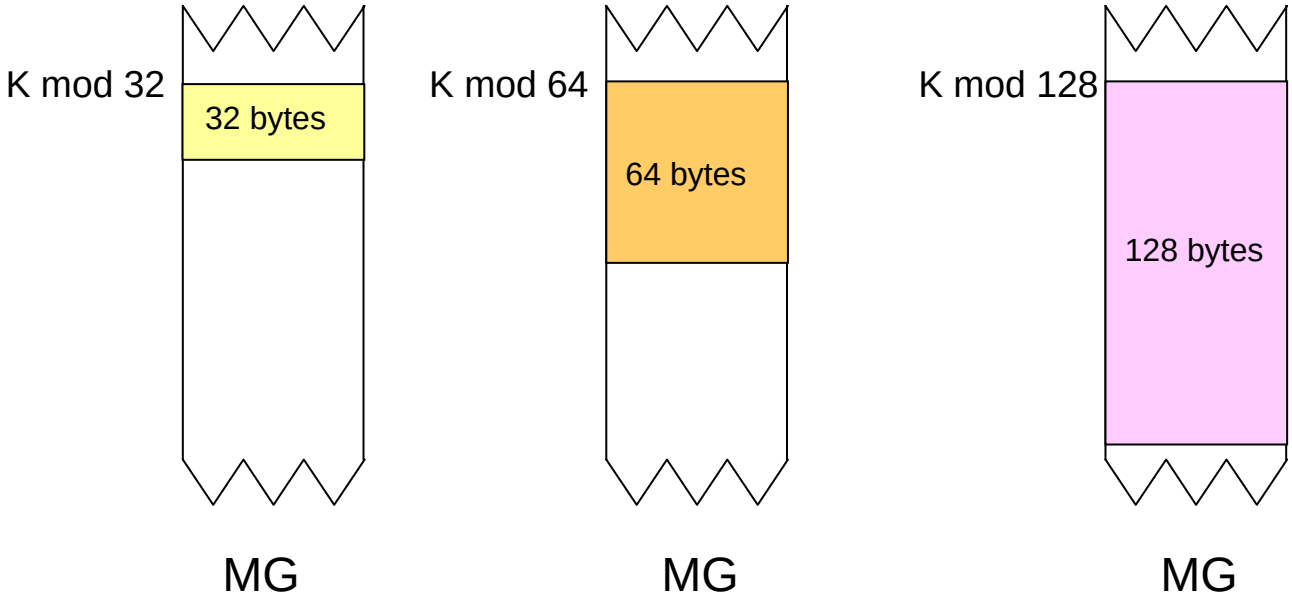
Memoria off-chip



- Accessible a todos los SMs (threads) y al host
  - Memoria global (R/W)
  - Memoria constante (R)
  - Memoria de textura (R/W)
- Contenidos son mantenidos entre ejecuciones de distintos grids
  - Intercambio de información entre threads de grids diferentes

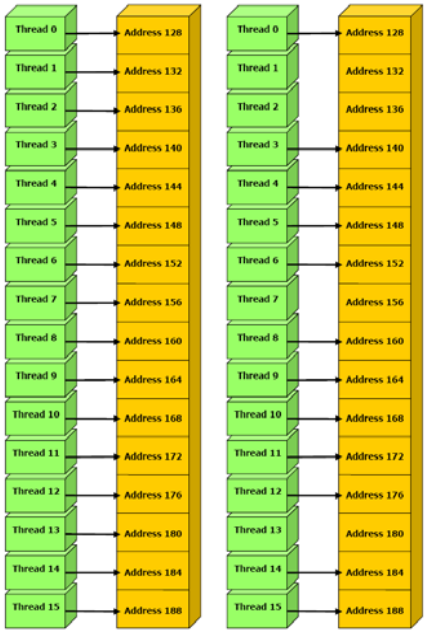
Coalescing en memoria global

- Instrucciones pueden leer/escribir palabras de 32, 64 o 128 bits
- Medio warp puede acceder de forma unificada (coalesced) para leer 32, 64 o 128 bytes a segmentos de memoria global
  - Segmento: Trozo contínuo de memoria de 32 , 64 o 128 bytes, alineado a estos tamaños

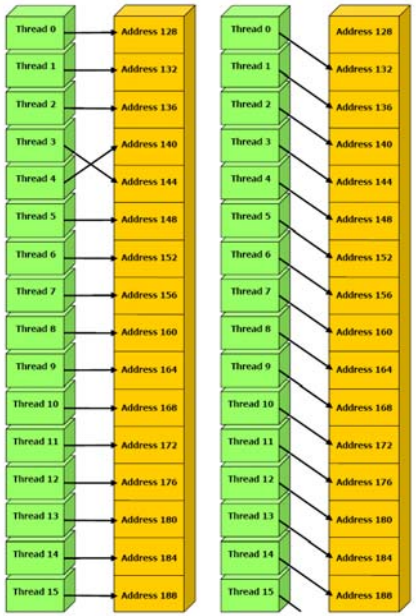


Acceso a la memoria global

- Condiciones de *coalescing* de threads con *Computing Capability* 1.0 y 1.1
  - Acceso a palabras de 32 bits (1 acceso de 64 bytes), de 64 bits (1 acceso de 128 bytes) o de 128 bits (2 accesos de 128 bytes)
  - Las 16 palabras deben estar ubicadas en un segmento de memoria de igual tamaño al de la transacción (o del doble en el caso de 128 bits)
  - Los threads deben acceder a las palabras en secuencia (k-ésimo thread a la k-ésima palabra)
- Si las condiciones no se cumplen se requiere un acceso independiente para cada thread

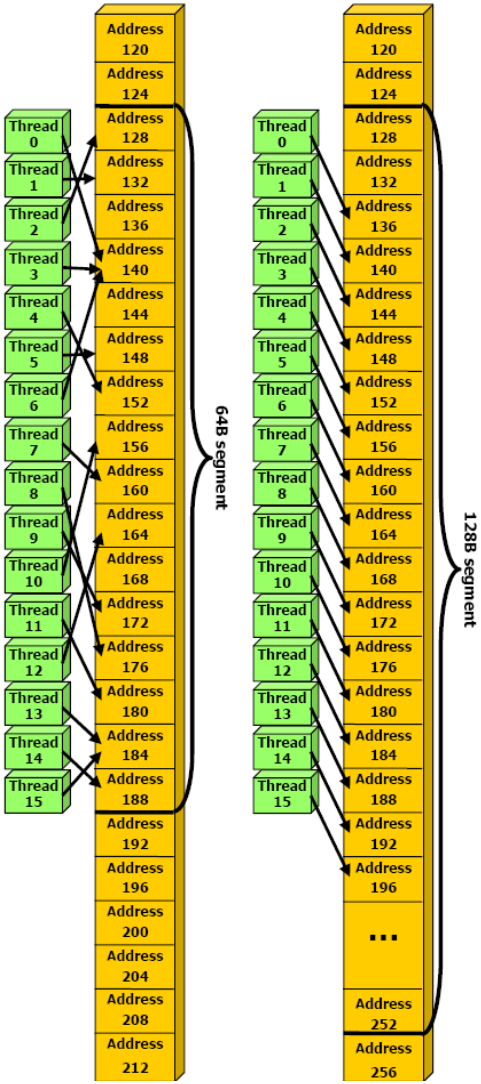


Coalesced



No-coalesced

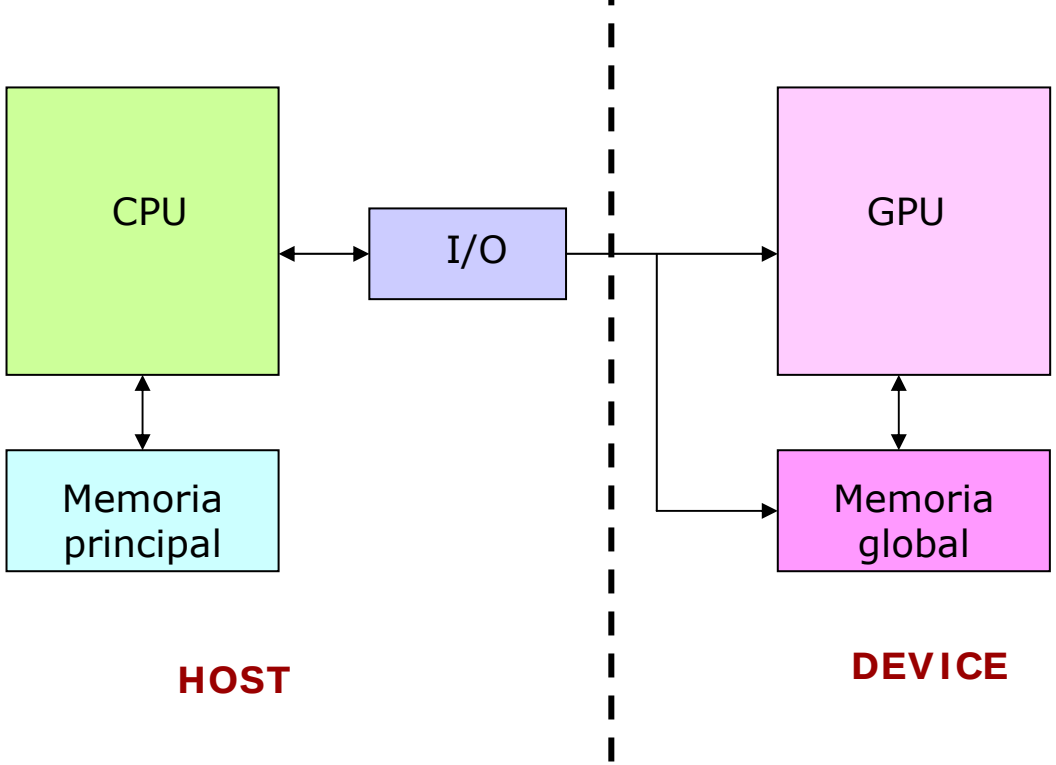
Acceso a la memoria global



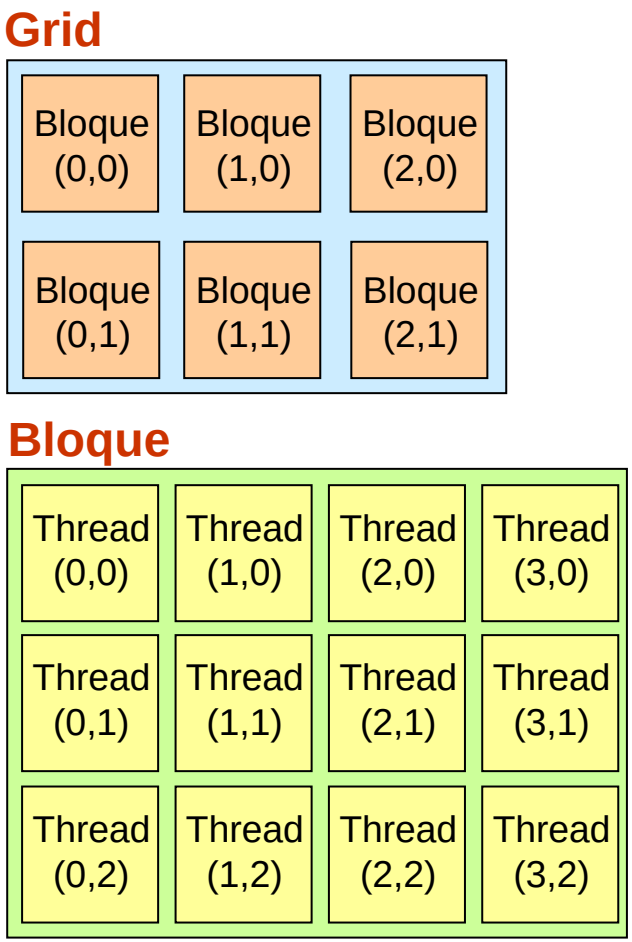
- Condiciones de coalescing para Computing Capability 1.2 o mayor
  - Palabras accedidas por lo threads están ubicada en un segmento de memoria de 32 bytes (palabras de 8 bits), 64 bytes (16 bits), 128 bytes (32 o 64 bits)
  - El patrón de acceso puede ser cualquiera e incluso varios threads pueden acceder a la misma palabra
  - Si el medio warp accede a n segmentos, el número de accesos será n

¿ Qué es CUDA ?

- CUDA ejecuta sobre un ***device*** físico separado que opera como coprocesador de un ***host***
  - Extensión de C
  - Biblioteca de *runtime con tres tipos de componentes:*
    - *Host: control y acceso a los devices*
    - *Device: funciones específicas de los devices*
    - *Comunes: tipos para vectores y un bibliotecas soportadas por el host y el device*



Jerarquía de thread



- Grid:
  - Cada kernel de ejecución del programa se mapea en un grid
    - Explicitar número de bloques en el grid
- Bloque:
  - Un bloque está compuesto de threads
    - Explicitar número de threads del bloque
  - Threads pueden cooperar compartiendo datos de la *shared memory*
  - Threads pueden sincronizarse (barrier)
- Warp:
  - Grupo de 32 threads consecutivos de un bloque
  - Tamaño de warp implícito

Modelo de programación

- Extiende lenguaje C con un nuevo tipo de función de tipo *kernel*
  - *Código de esta función es ejecutado en paralelo por múltiples threads en un device (GPU)*

Ejecución

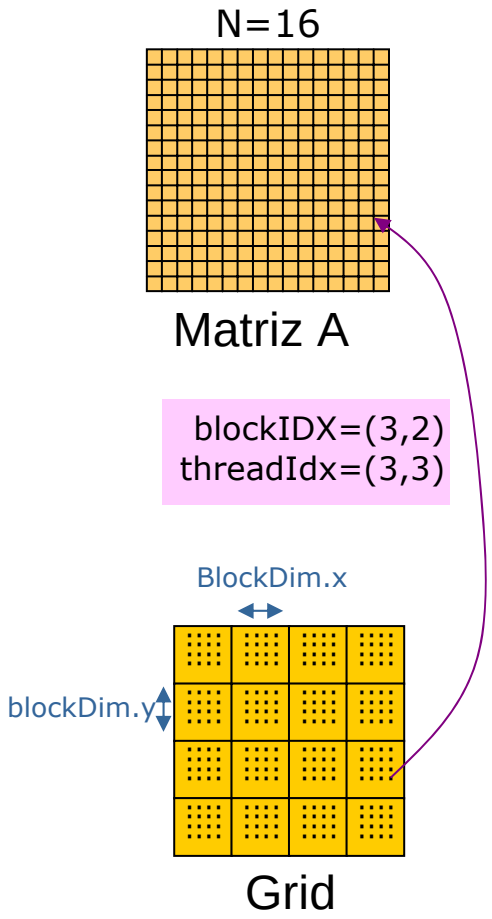
```
__global__ kernelA(){···}
__global__ kernelB(){···}
int main()
...
kernelA <<<dimGridA,  dimBlockA>>>;
...
kernelB <<<dimGridB, dimBlockB>>>
...
```

CPU
GPU
CPU
GPU
CPU

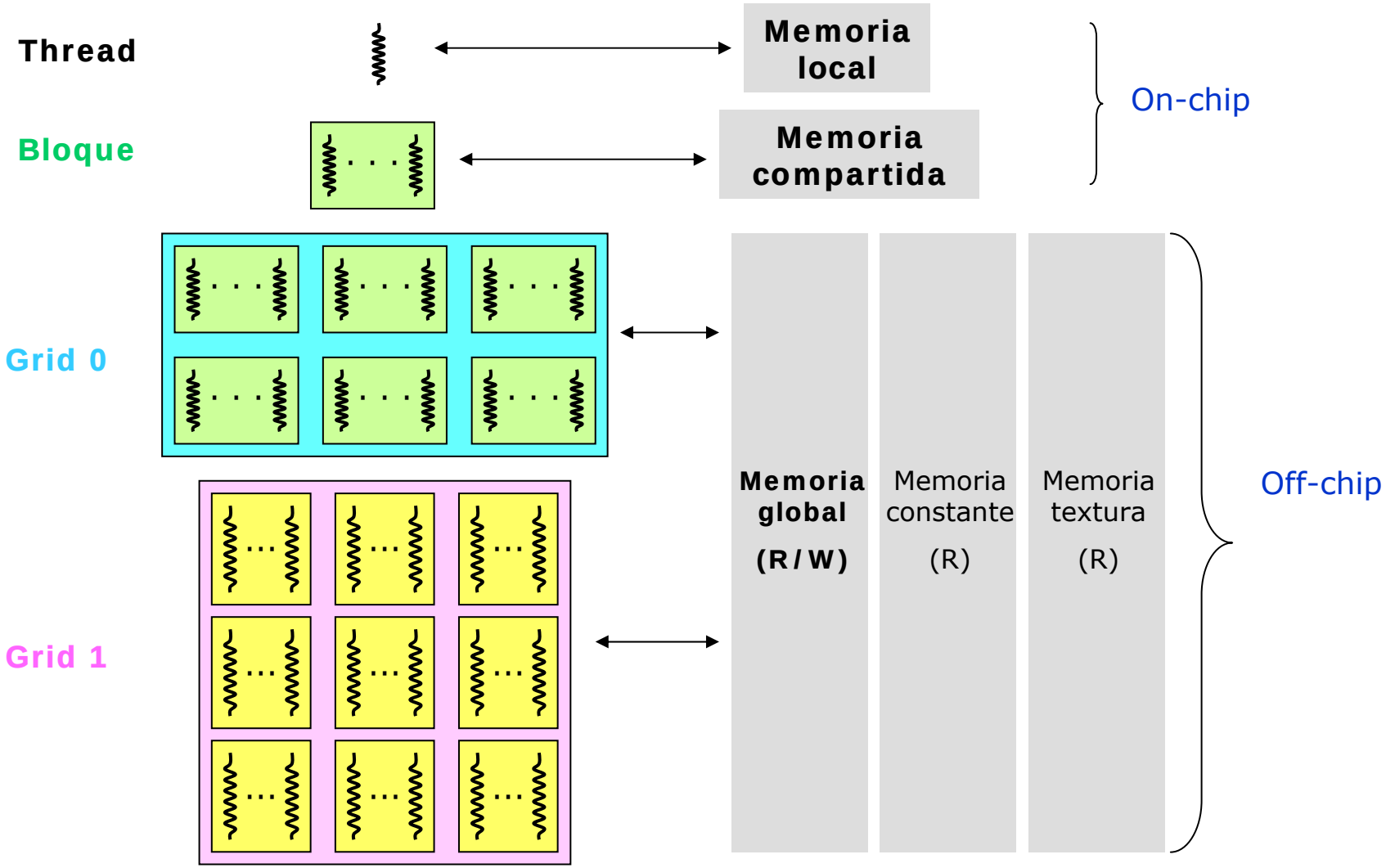
Identificación de threads

- Bloque
  - BlockIdx (*built-in*): vector (1D, 2D o 3D) que identifica el bloque dentro del grid.
- Thread
  - ThreadIdx(*built-in*): vector (1D, 2D o 3D) que identifica el thread dentro del bloque

```
__global__ void matAdd (float A[N][N], float B[N][N],
                        float C[N][N])
{
   int i = blockIdx.x * blockDim.x + threadIdx.x;
   int j = blockIdx.y * blockDim.y + threadIdx.y;
   C[i][j]=A[i][j]+B[i][j]
{
int main(){
   dim3 dimBlock(4,4);
   dim3 dimGrid (N/ dimBlock.x, N/ dimBlock.y);
   matAdd<<<dimGrid, dimBlock)>>>(A, B, C);
}
```



Acceso a memoria



Control de flujo

- Cada SM ejecuta los 32 threads de un warp (SPMD)
  - Todos los threads ejecutan la misma instrucción
- Si bifurcaciones afectan de forma diferente a los threads de un warp (divergencia), se secuencializa la ejecución de los threads del warp
  - Pérdida de rendimiento

```
if (ThreadIdx.x > 2)
{
}
```

Divergencia

```
if (ThreadIdx / WARP_SIZE >2)
{
}
```

Sin Divergencia

Instrucciones predicadas

- Transformar la dependencia de control en una de datos
  - En el caso de la GPU evitamos el problema de la divergencia
- Idea interesante si hay pocas instrucciones en el *if-else*

```
if (x == 10)
        c=c+1;
```

```
        ld r5, X
        P1 <- r5 eq 10
P1:     ld r1, c
P1:     add r1, r1, 1
P1:     st r1, c
```

Predicados en CUDA

- La instrucciones predicadas permiten la escritura si la condición se cumple
  - Se evitan los accesos a memoria (*loads* y *stores*) de instrucciones que no cumplen condición
- El compilador analiza el posible grado de divergencia de warps
  - Si deduce que no hay divergencia sólo aplica predicados si hay menos de 4 instrucciones
  - Si no hay garantías predica con menos de 7 instrucciones

Coalescencing en 1-D arrays

- Acceso a arrays de estructuras
  - Si tamaño estructura mayor de 16 bytes, crear varios arrays de estructuras de este tamaño como máximo

```
struct{
   float var1;
   float var2;
   . . .
   float var8;
} big;
```

struct big[10000];

```
struct{
   float var1;
   . . .
   float var4;
} half1;
```

struct half1[10000];

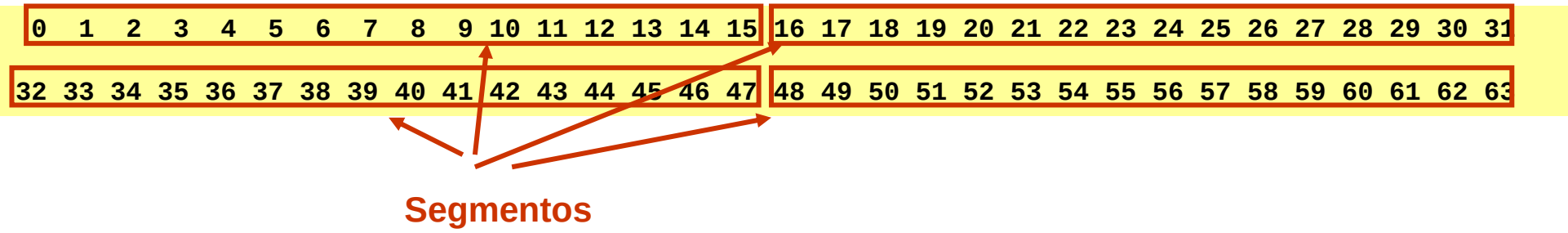
```
struct{
   float var5;
   . . .
   float var8;
} half2
```

struct half2[10000];

Coalescencing en arrays 2-D

- En arrays de estructura 2-D con acceso a elementos cotigüos de la fila:
  - Ancho de la fila debe ser múltiplo de 16 (por alineamiento)

```
float array[2][16]
```



Reducción sin conflictos en SM

- Acceso de cada thread activo a módulos diferentes en el mismo ciclo

```
float A[64];
```

```
load r1, A[tid];
load r2, A[tid+32];
add r2,r2,r1
store r2, B[tid];
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63

```
// Sólo si tid <16
   load r1, B[tid];
   load r2, B[tid+16];
   add r2,r2,r1
   store r2, B[tid];
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

```
// Sólo si tid <8
   load r1, B[tid];
   load r2, B[tid+8];
   add r2,r2,r1
   store r2, B[tid];
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

```
// Sólo si tid <4
   load r1, B[tid];
   load r2, B[tid+4];
   add r2,r2,r1
   store r2, B[tid];
```

0	1	2	3	4	5	6	7

Reducción y divergencia

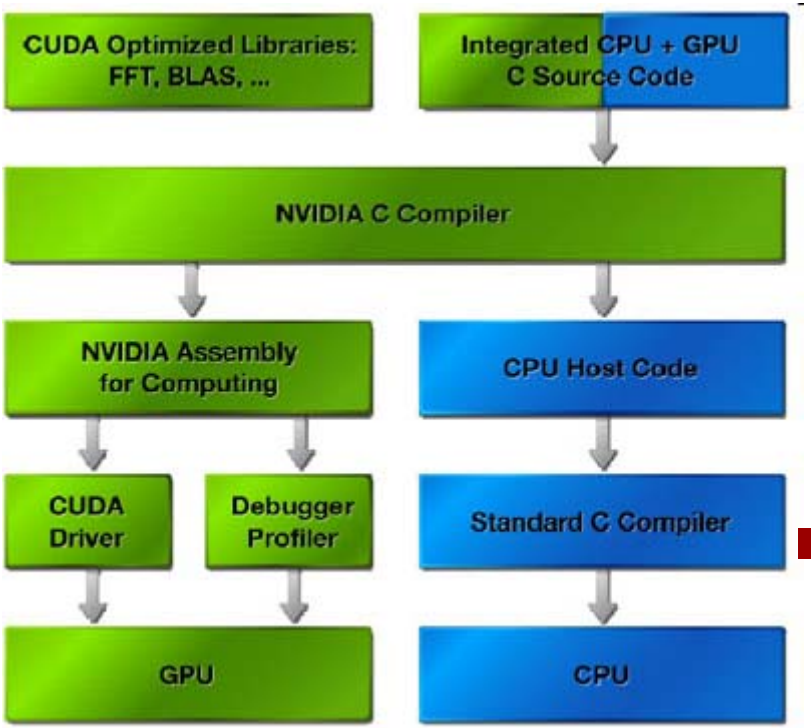
- Divergencia presente cuando stride<=16
  - Sólo threads con tid < 16, 8, 4, y 2 computan

```
__shared__ float partialSum();

unsigned int t= threadIdx.x;
for (unsigned int stride = blockDim.x;
     stride >1; stride >>1)
{
  __syncthreads();
  if (t < stride)
    partialSum[t] += partialSum[t+stride];
}
```

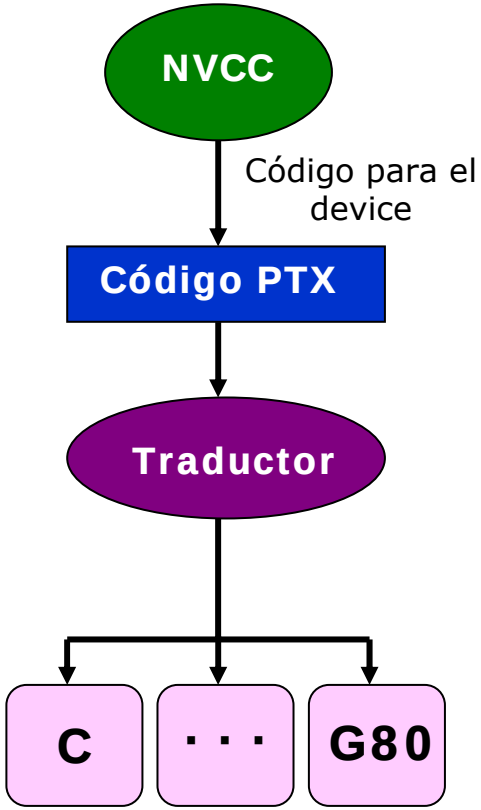
- No hay conflictos en acceso a la SM

Compilando con NVcc



- NVcc genera dos tipos de código:
  - Host: donde finalmente se usará un compilador de la CPU para generar el código final
  - Device: código PTX que incluye los kernel y funciones llamadas por ellos
- Los dos códigos serán enlazados (*binded*) por el runtime
  - Comunicaciones entre estos códigos será realizada por el driver y la biblioteca de transferencia

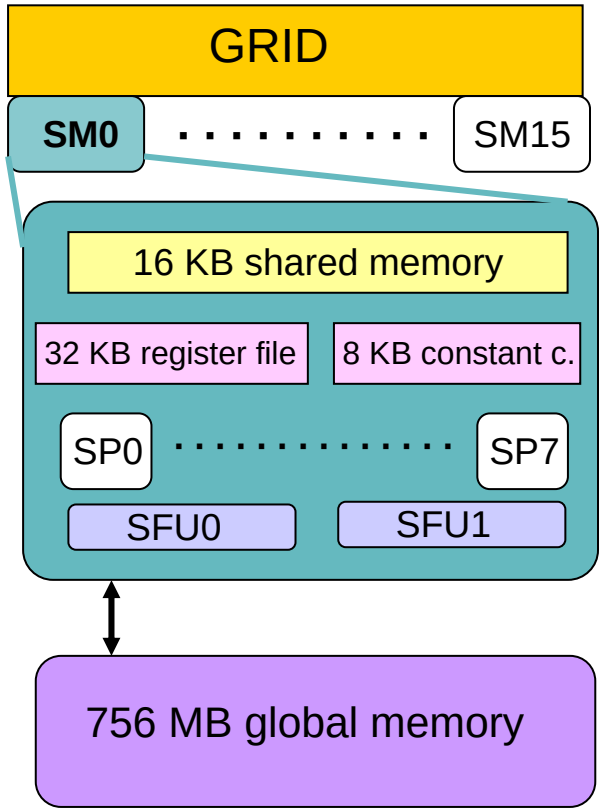
PTX (Parallel thread execution)



- Código para máquina virtual
  - Basado en ISA
  - Expone todos los recursos computacionales
- Traductor
  - Replanifica y optimiza ligeramente el código en función de la arquitectura real durante el runtime

Ejemplo
Multiplicación de matrices

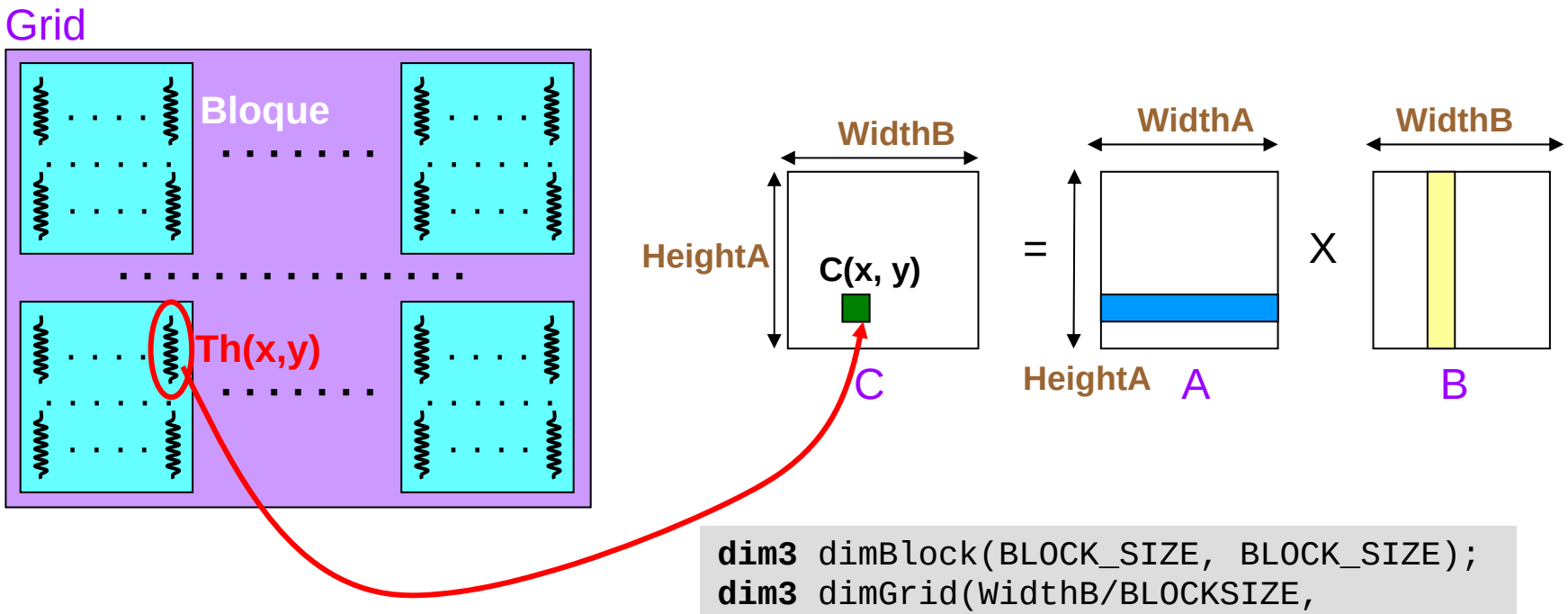
Arquitectura G80



- 388.8 GFlops
  - 16 SMs * 18 FLOPS/SM * 1.35 GHz
- Memoria global:
  - 364 bits
  - 84.6 GB/s

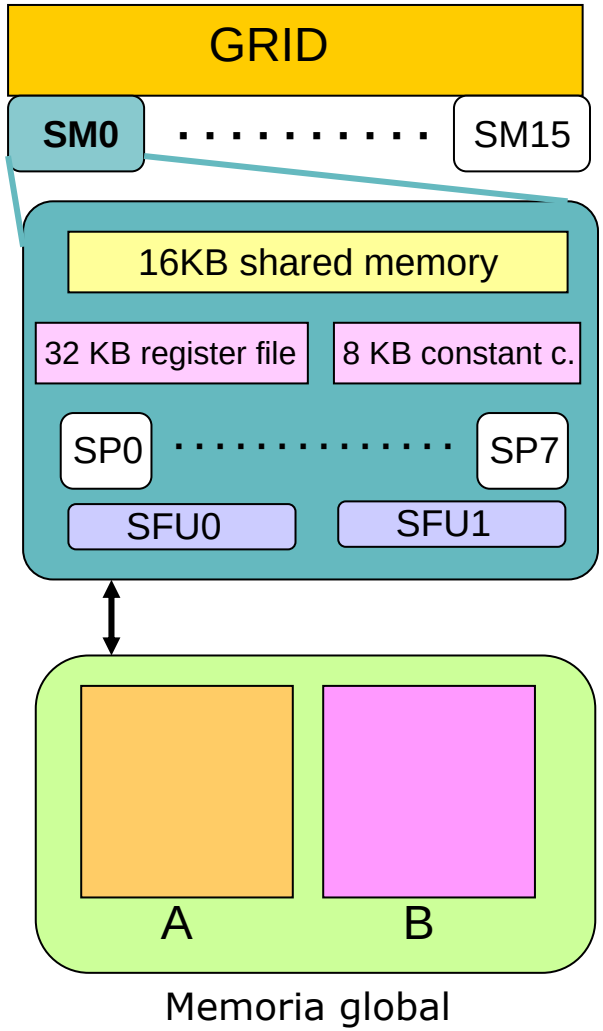
Organización del computo

- Recomendable crear gran cantidad de threads para ocultar latencias con la memoria (evitando sincronizaciones)
  - Usar un sólo grid con tantos threads como elementos haya en la matriz C
  - Cada thread se encargará de realizar el cómputo de un elemento de la matriz C




```
dim3 dimBlock(BLOCK_SIZE, BLOCK_SIZE);
dim3 dimGrid(WidthB/BLOCKSIZE,
                  HeightA/BLOCKSIZE);
```

Versión simple




```
Ctemp=0;
for (i=0; i<widthA; i++){
   Ctemp += A[indexA] * B[indexB];
   indexA++;
   indexB += width
}
C[indexC] = Ctemp ;
```

Coalescing en acceso a matriz B

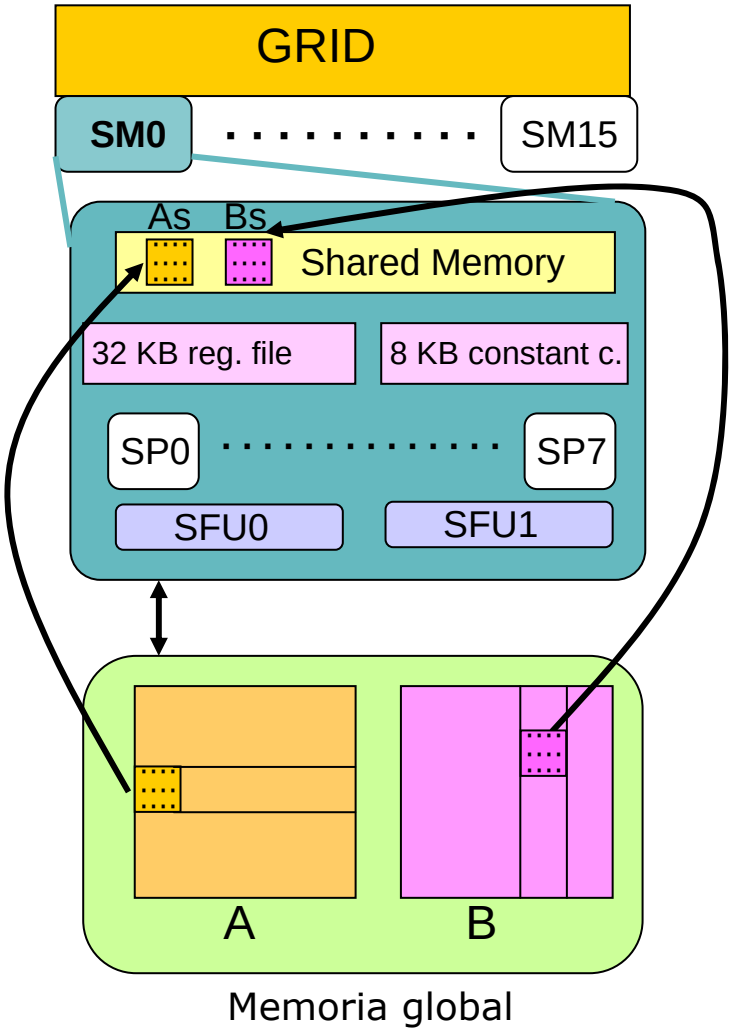
10 registros por thread ->768 threads: SM agrupados en tres bloques de 256

Rendimiento: 10.58 GFLOPS

Cuello de botella (mirar PTX) -> acceso a memoria global

Tiling en memoria compartida

- Cada thread carga un elemento de a y b del tile




```
Ctemp=0;
for (···){
   __shared__ float As[16][16];
   __shared__ float Bs[16][16];

   // Load tile (16x16)
   As[ty][tx] = A[indexA];
   Bs[ty][tx] = B[indexB];
   indexA += 16;
   indexB += 16 * widthB;
   __syncthreads();

   // Compute results from tile
   for (i=0; i<16; i++)
         Ctemp+=As[ty][i]*Bs[i][tx];
   __syncthreads();
}
C[indexC] = Ctemp ;
```

Desenrrolle

- Optimización automática del compilador

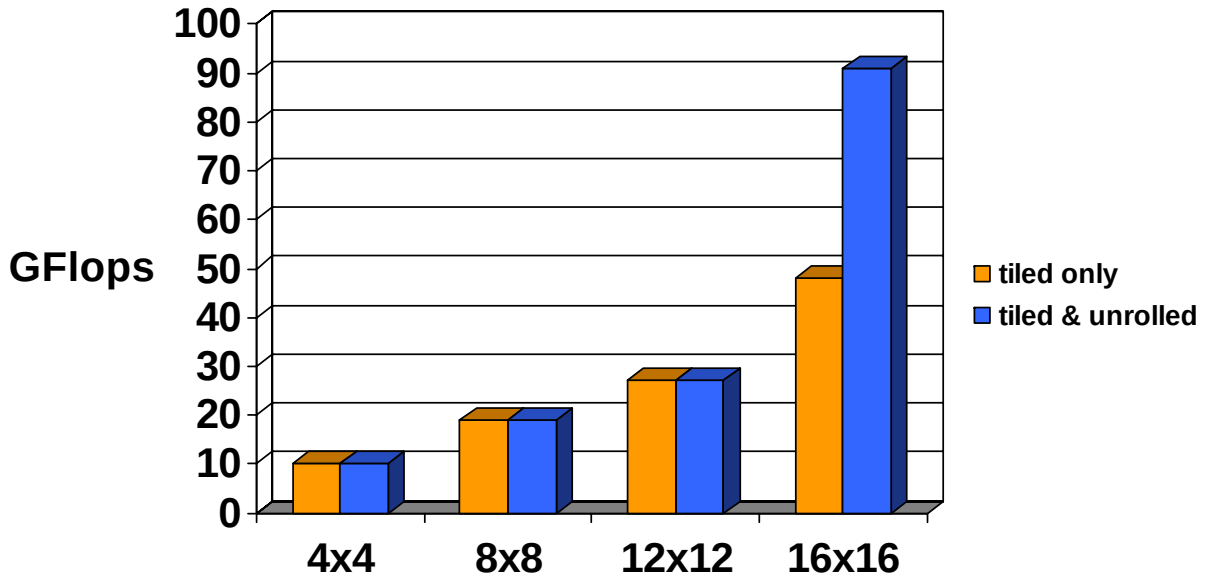
```
Ctemp=0;
for (···){
   __shared__ float As[16][16];
   __shared__ float Bs[16][16];

   // Load tile (16x16)
   As[ty][tx] = A[indexA];
   Bs[ty][tx] = B[indexB];
   indexA += 16;
   indexB += 16 * widthB;
   __syncthreads();

   // Compute results from tile
   Ctemp+=As[ty][0]*Bs[0][tx];
         ....
   Ctemp+=As[ty][15]*Bs[15][tx];
   __syncthreads();
}
C[indexC] = Ctemp ;
```

Tamaño del tile

- Rendimiento



Conclusiones

- GPU puede ser usada como dispositivo acelerador de código basado en kernels
  - Eficiencia basada en la ejecución concurrente de miles de threads
  - Gran número de unidades funcionales
- Optimización de código compleja
  - Optimizar accesos a memoria global
  - Gestión explícita de la memoria compartida
  - Ajuste fino de tamaño de bloque
- Aspectos no visibles
  - Asignación y planificación de threads